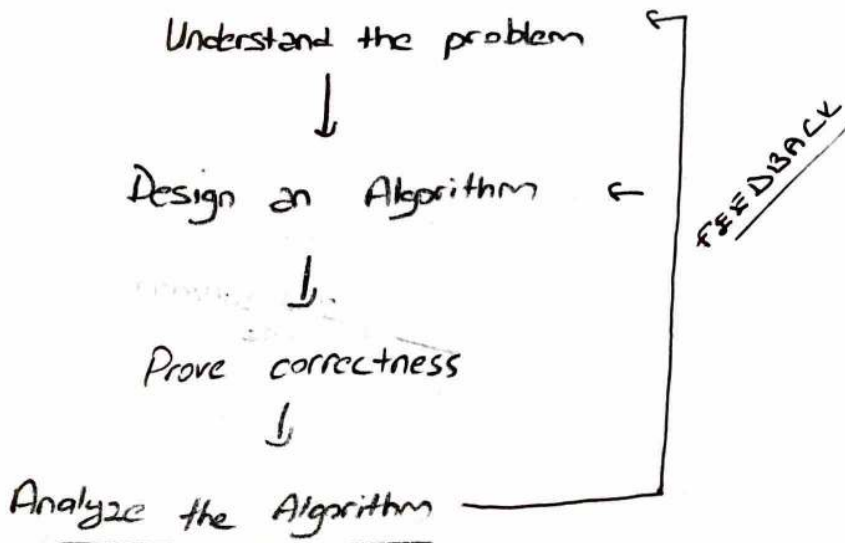


Veri Yapıları ve Algoritmalar :

Kitap = Algorithms 4th edition = Robert Sedgwick

Des1 :



① efficiency $\begin{cases} \rightarrow \text{time} \\ \rightarrow \text{space} \end{cases}$

② simplicity

③ generality

Sieve of Eratosthenes :

→ verilen N sayısı için 2'den N 'e kadar olan asal sayıların bulunması

$N = 25$

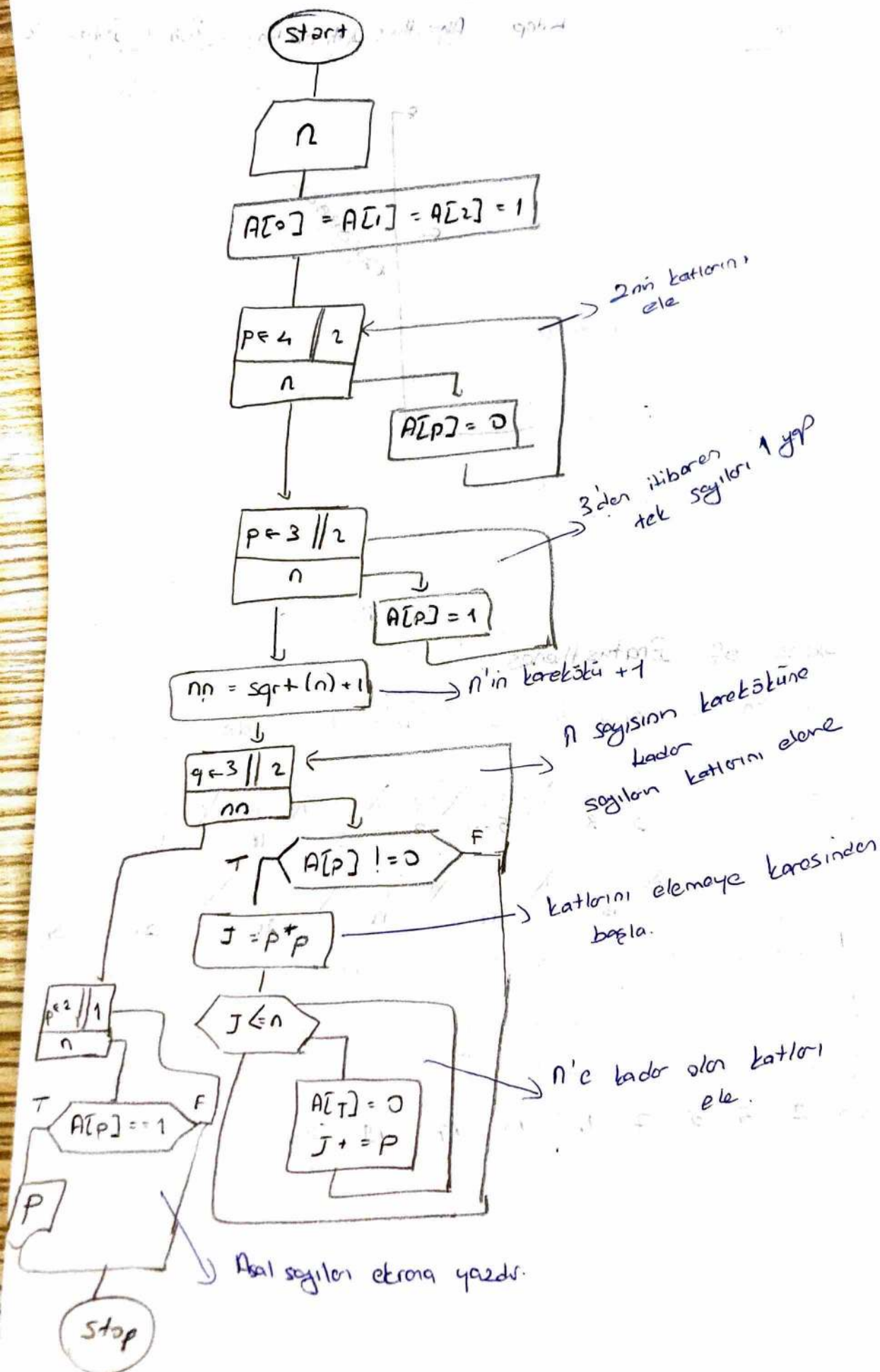
2	3	4	5	6	7	8	9	10	11	12	13
2	3		5		7		9		11		13
		14	15	16	17	18	19	20	21	22	23
		15			17		19		21		23
											24
											25

Kontrol → sayının kendisine kadar herhangi birine bölünüyor mu?

→ Ya da katları olan sayıları eleme → daha efektif
 $N = 25$ $\sqrt{25} = 5$ en son 5 in katlarını eleme yeterli

↳ 2 3 5 7 11 13 17 19 23

Sieve Algorithm :



→ Merging two sorted Arrays

(İki dizi de sıralı)

a →

0	1	2	3	4	5
3	5	7	10	12	16

$N_a = 6$

b →

1	4	18	21
---	---	----	----

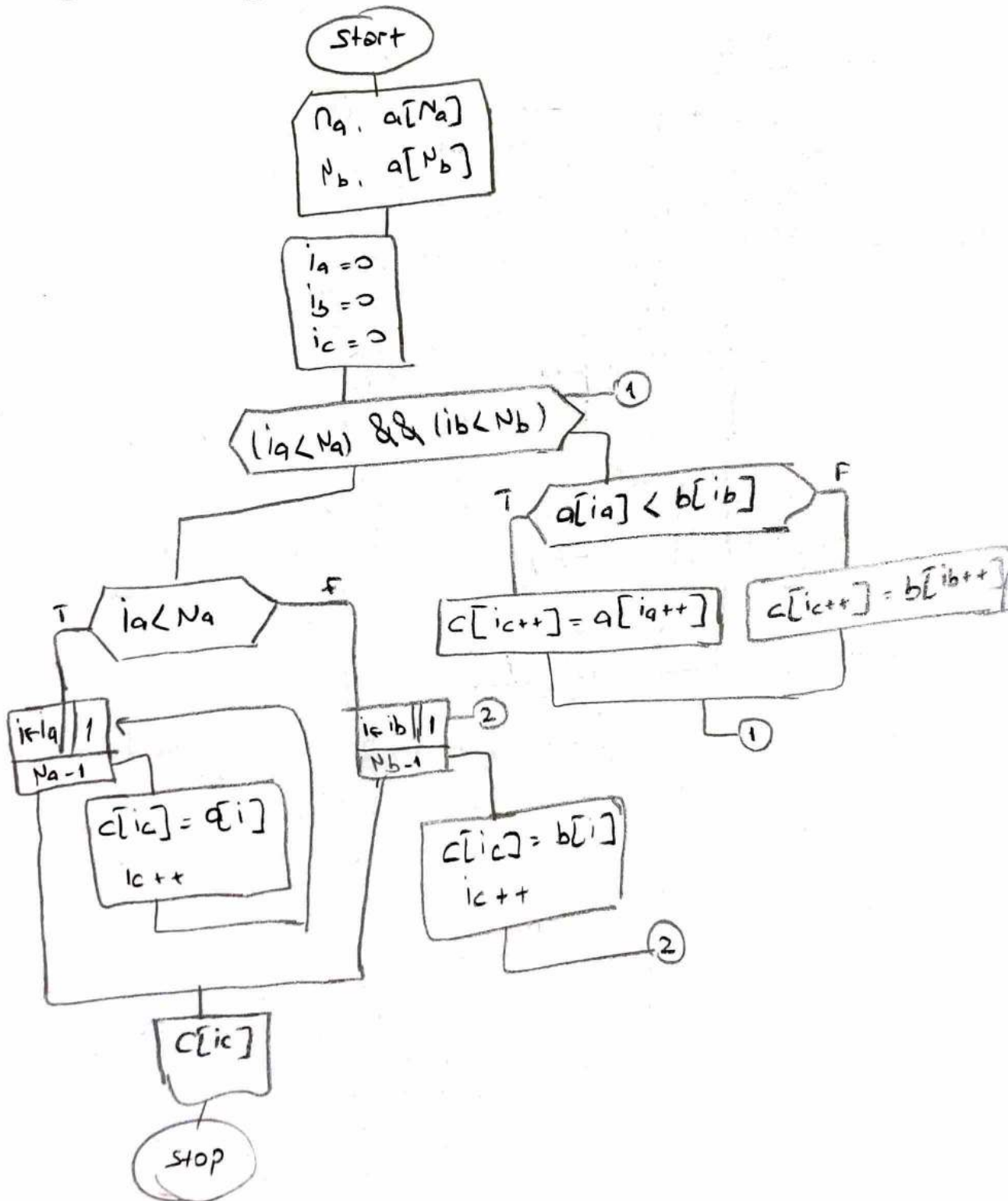
 $N_b = 4$

0	1	2	3	4	5	6	7	8	9

→ İki dizinin de sıralı olduğu avantajını kullanan

→ dizinin ilk elemanlarını karşılaştır

→ bir dizi bittiği zaman dur.



2. Ders :

Sıralama Algoritmaları :

Bubble Sort :

0 1 2 3 4
a = [6, 2, 11, 7, 5]

2 6 11 7 5
2 6 11 7 5

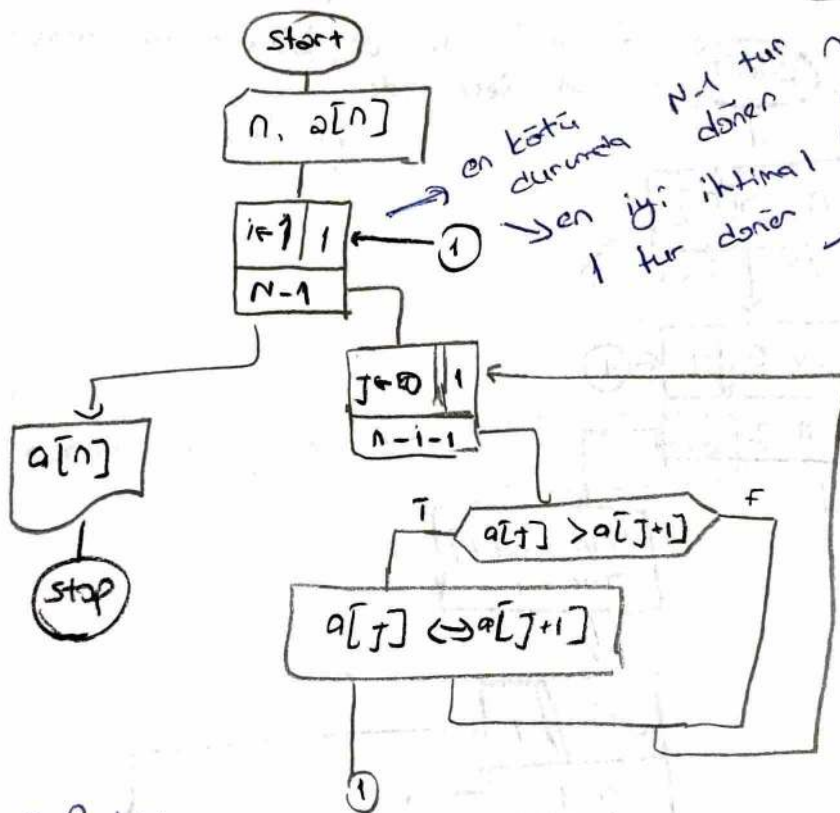
→ 2 6 7 11 5

→ 2 6 7 5 11

→ en büyük sayı en sona taşındı

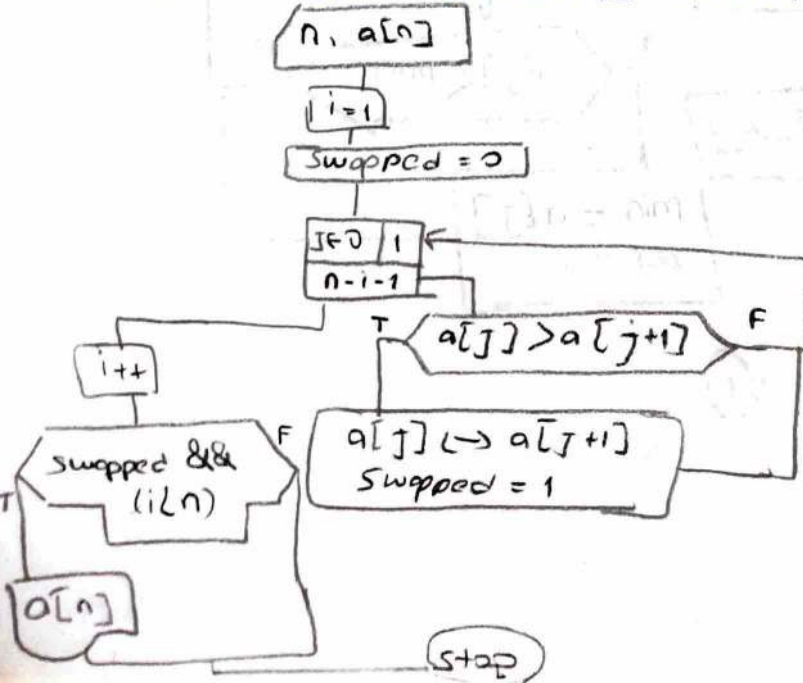
tekrar → 2 6 5 7 11

2 5 6 7 11



• Zaman
dönü
sayısının
belli olması
efektif değil

→ Bubble Sort Effective Version :



Selection Sort :

0	1	2	3	4	5	6
2	7	4	9	3	1	8

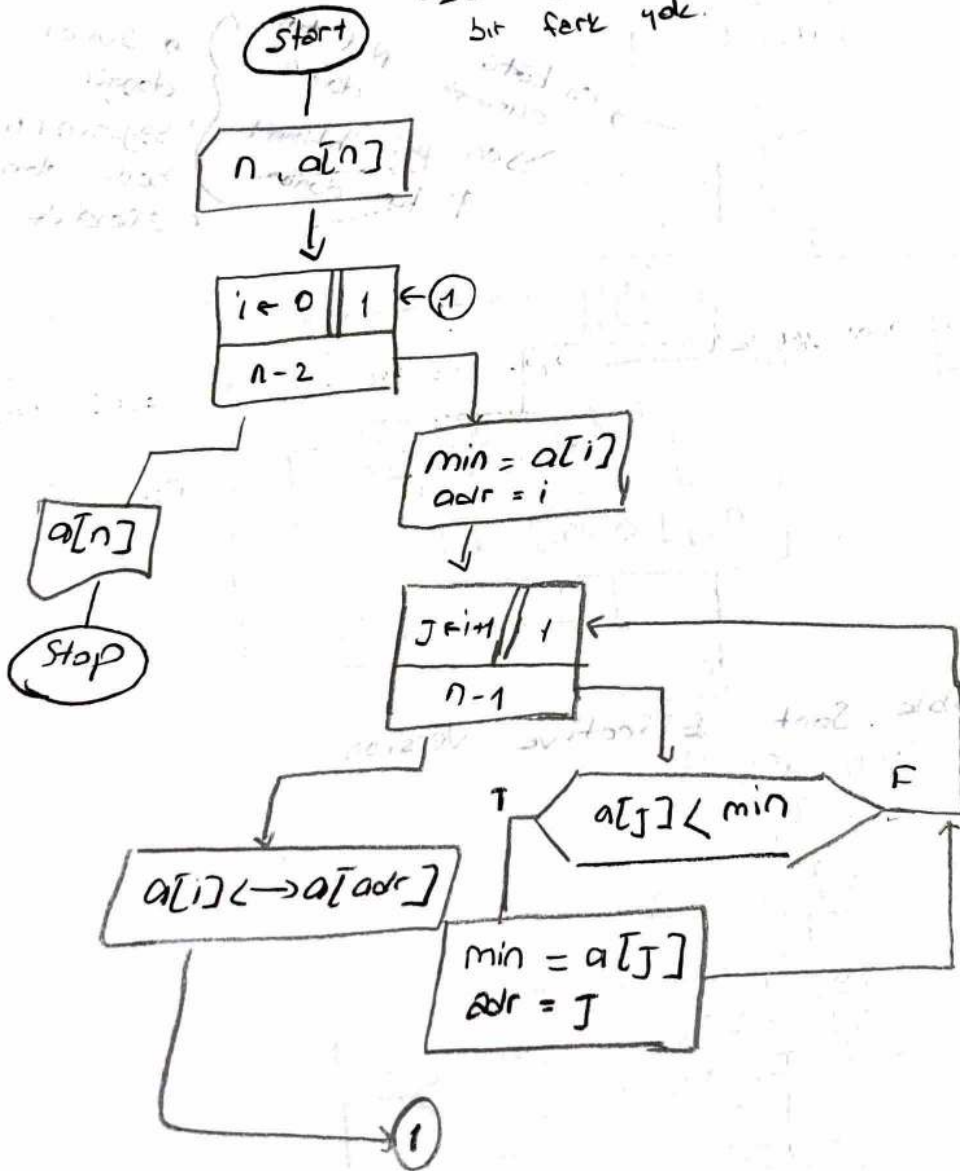
→ 1 7 4 9 3 2 8 → 1 2 4 9 3 7 8

1 2 3 4 7 9 8

← 1 2 3 4 9 7 8 ← 1 2 3 9 4 7 8

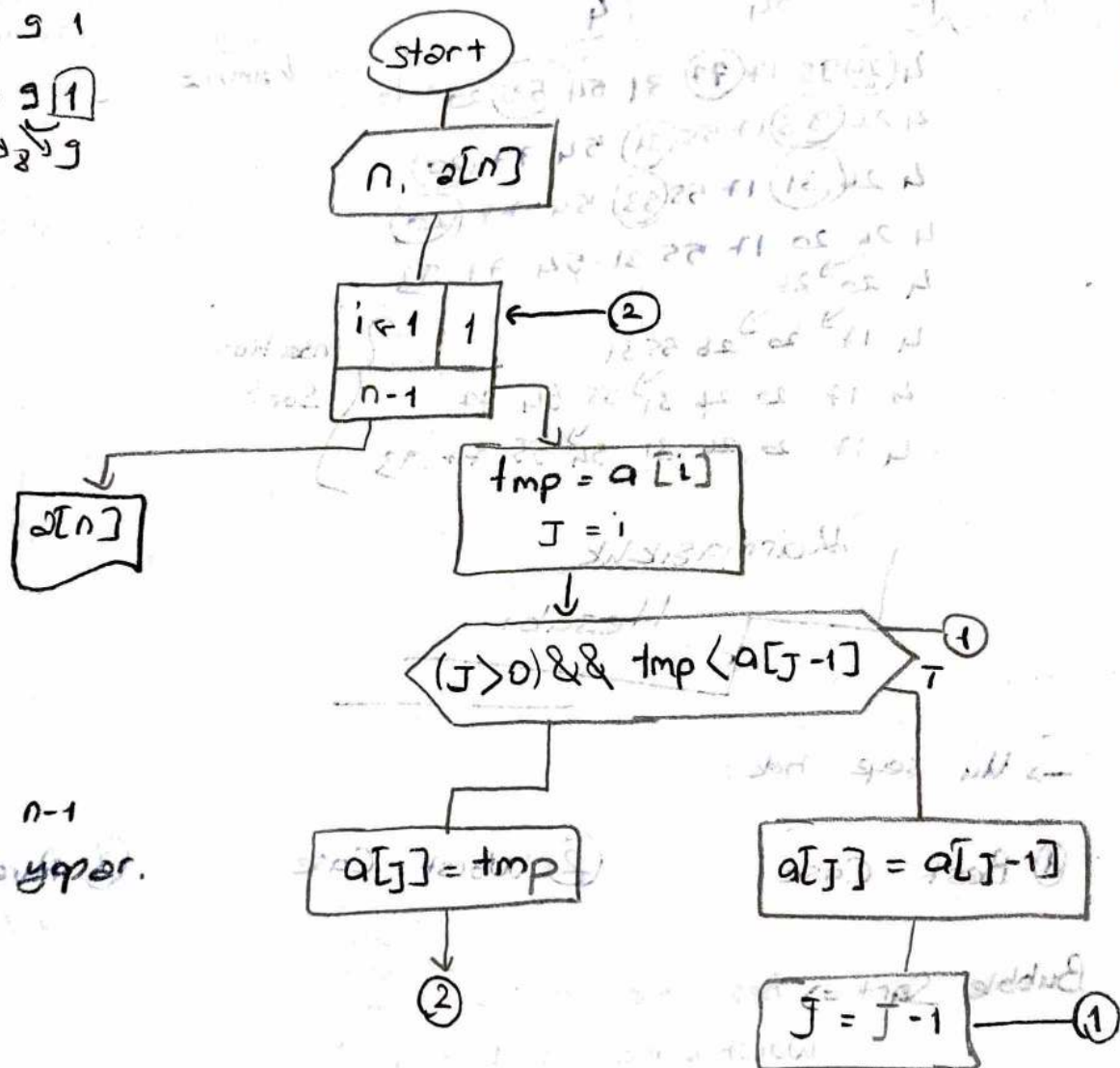
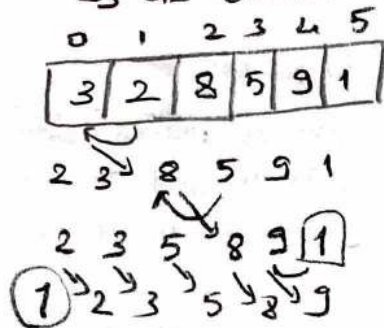
1 2 3 4 7 8 9

Dezenkontaj → dışı sıralı da olsa sonuna kadar değiştirme işlemlerini kontrol eder
→ en iyi ve en kötü durum arasında bir fark yok.



Insertion Sort :

→ a2 elementi diziler iuin en efektif siralama algoritması.



→ dizi siralıysa $n-1$ karşılaştırma yapar.

3. Ders:

① How to analyze algorithm?

1- Compare execution times

⇒ System dependents effects:

- Hardware: CPU, memory, cache ..
- Software: compiler, interpreters
- System: OS, network other apps

⇒ System independent effects:

- Algorithm
- Input (girilen sayıların sırası örn)

Bu etkenlerle nasıl efektif algoritma analizi yapabiliriz?

② Express running time as a function of the input size N

Örn: → çalışma zamanını N 'e bağlı olarak söyleyelim:

```
int i, n; count=0;
```

```
for (i=0; i<n; i++) {
```

```
    if (a[i] == 0)
        count++;
```

```
}
```

=

operation

frequency

Variable dec.

3 (deklarasyon)

Assignment stat.

2 (atama)

less than compare

$N+1$

equal to compare

N

array access

N

increment

N to $\frac{2N}{\text{worst}}$

$\frac{N}{\text{best}}$
 $4N+6$ to $5N+6$

Order of Growth:

growth rate of algorithm.

$4N+6$ $N \rightarrow \infty$

1 → ignore coefficients $\Rightarrow N+6$

2 → ignore constants $\Rightarrow N$

3 → pick the most significant term $3N^2+2N+5 \rightarrow \underline{N^2}$

Asymptotic Analysis ⇒ bir algoritmanın karmaşıklık hesabını hardware, Software den bağımsız hesaplaması

Asymptotic Notation $\Rightarrow$ Big - Oh = $O()$ $\rightarrow$ upper bound
 Big - theta = $\Theta()$ $\rightarrow$ tight bound
 Big - omega = $\Omega()$ $\rightarrow$ lower bound

Classes of Algorithms:

- Constant time = $O(1)$
- Logarithmic time = $O(\log_2 n)$ (Örn: 2'ye bölerek giden algoritma)
- Linear time = $O(n)$ (Örn: for(i=0; i<n; i++)
- Linearithmic time = $O(n * \log_2 n)$ (Örn: Quick sort, merge sort)
- Quadratic time = $O(n^2)$ (Örn: double loop)
- Exponential time = 2^N (gerçek zamanda çalışmaz)

ÖRN :

int sequentialSearch(int n, int a[], int x)

{

 i=0;

 while((i<n) && (a[i] != x))

 i++;

 if (i<n)

 return i;

 else

 return -1;

}

1- en çok zaman alan işi bul.

1- Worst case efficiency = $O(n)$

2- Best case efficiency = $O(1)$

3- Average case efficiency =

probability of successful search = p $0 \leq p \leq 1$

probability of first match occurring in the
 ith position = $\frac{p}{n}$

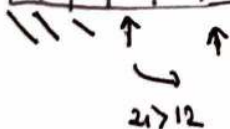
$$C_{avg}(n) = \underbrace{\left[1 * \frac{p}{n} + 2 * \frac{p}{n} + \dots + n * \frac{p}{n} \right]}_{\text{Successful search}} + \underbrace{[n * (1-p)]}_{\text{unsuccessful search}}$$

ÖRN: Binary Search =

0	1	2	3	4	5	6	7
2	7	9	12	17	21	23	27

x=21

$\Rightarrow$

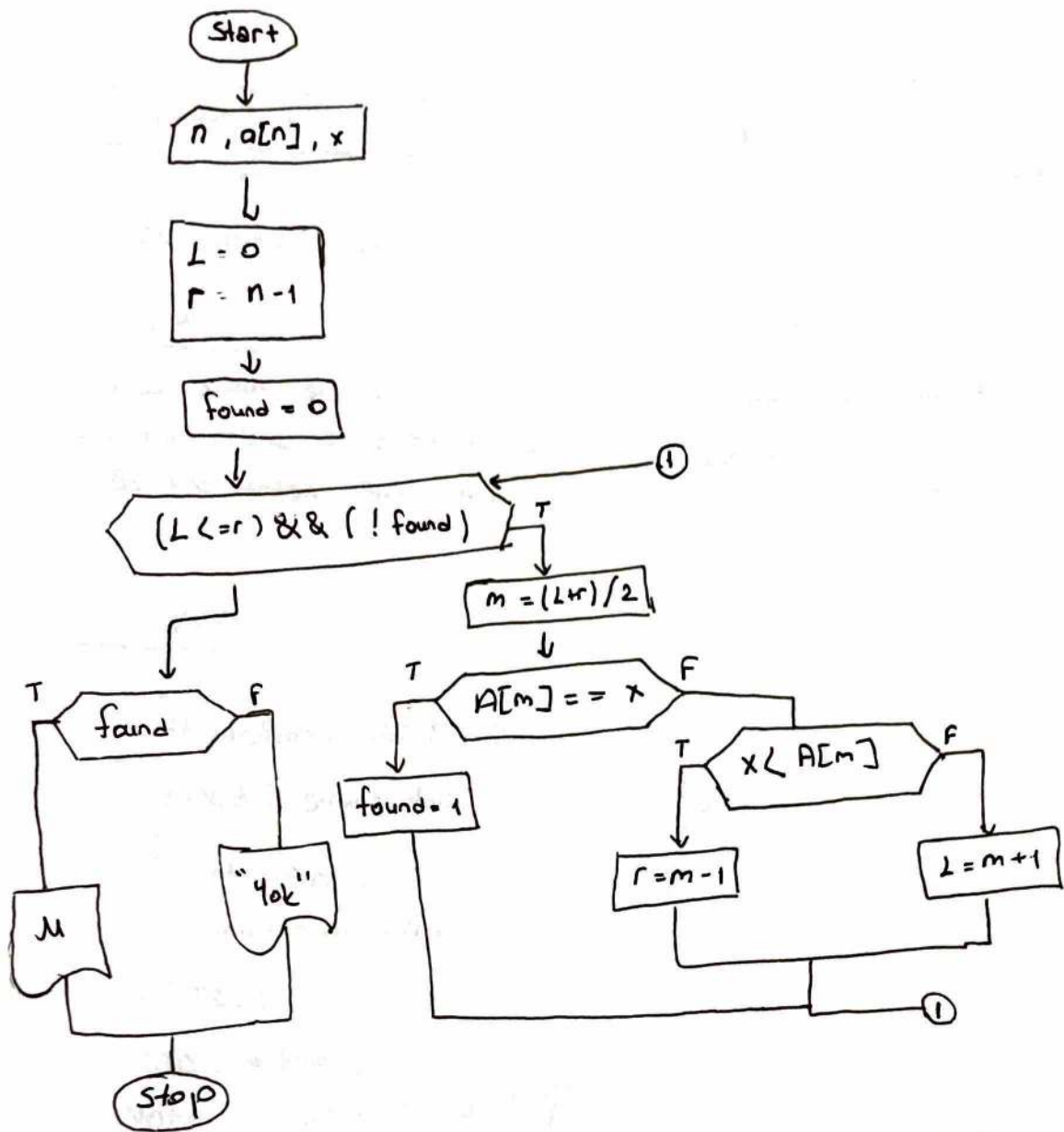


21 > 12

bas	son	orta	
0	7	3	21 > 12
4	7	5	21 = 21

$\rightarrow$ decrease and conquer algorithm

Binary Search Algorithm:



$$C_{\text{worst}}(n) = C_{\text{worst}}(n/2) + 1 \quad \text{for } (n > 1)$$

Compare

$$C_{\text{worst}}(n) = 1 \Rightarrow \text{if } n = 1$$

Backward
Substitution

$$C_w(n/4) + 1 + 1$$

$$C_w(n/2^i) + i$$

1

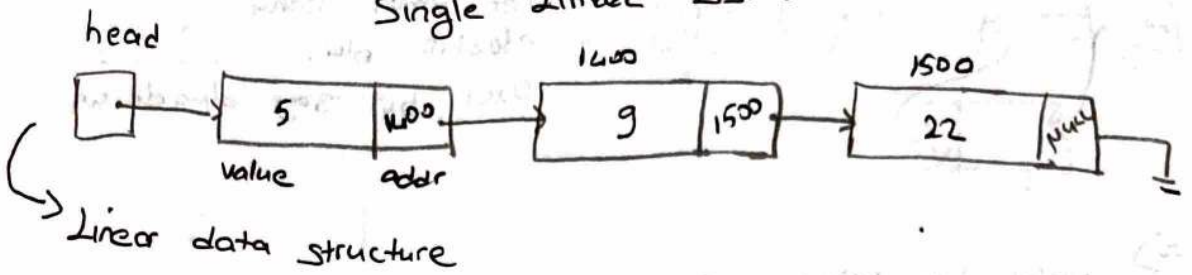
$$C_w(n) = C_w(1) + \log_2^n$$

$$= \log_2^n + 1$$

$$= \underline{\underline{O(\log_2^n)}}$$

Linked Lists :

Single Linked List :



Avantajlar

- Dizi uzunluğu sabit değil
- Eleman eklenir ve silme kolay
- Bellekte aralıksız bulunmama
-

Dezavantajlar

- Random access mümkün değil
- Pointer değişken için ekstra yer ihtiyacı var.

ÖRNEK KOD

```
#include <stdlib.h>

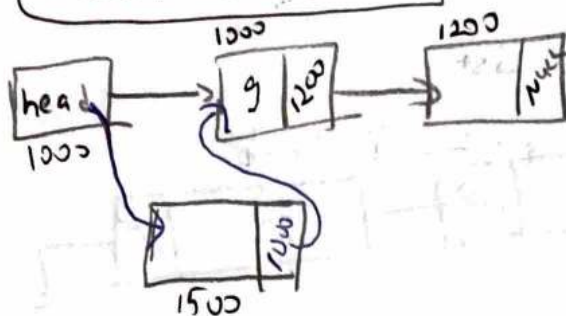
typedef struct Node
{
    int value ;
    struct Node * next ;
} NODE;

NODE * head;

NODE * create Node() {
    NODE * p;
    p = (NODE *) malloc (size of (NODE));
    if (p == NULL)
        exit(0);
    p->next = NULL;
    return p;
}

int main() {
    NODE * p;
    head = NULL;
    p = create Node();
    insert At Front (p, 8);
}
```

Insert at front



başta yerleştirmek
her zaman daha
etektif olur.
Özel bir say olmadıkça

=> void insertAtFront (NODE * newNode, int value)

```

{
    newNode->value = value;
    newNode->next = head;
    head = newNode;
}

```

Insert at End

=> void insertAtEnd (NODE * newNode, int value)

```

{
    NODE * tmp;
    if ( head == Null )
        head = newNode;
}

```

else

```

{
    tmp = head;
    while ( tmp->next != Null )
        tmp = tmp->next;
}

```

```

tmp->next = newNode;
}
}

```

void printList()

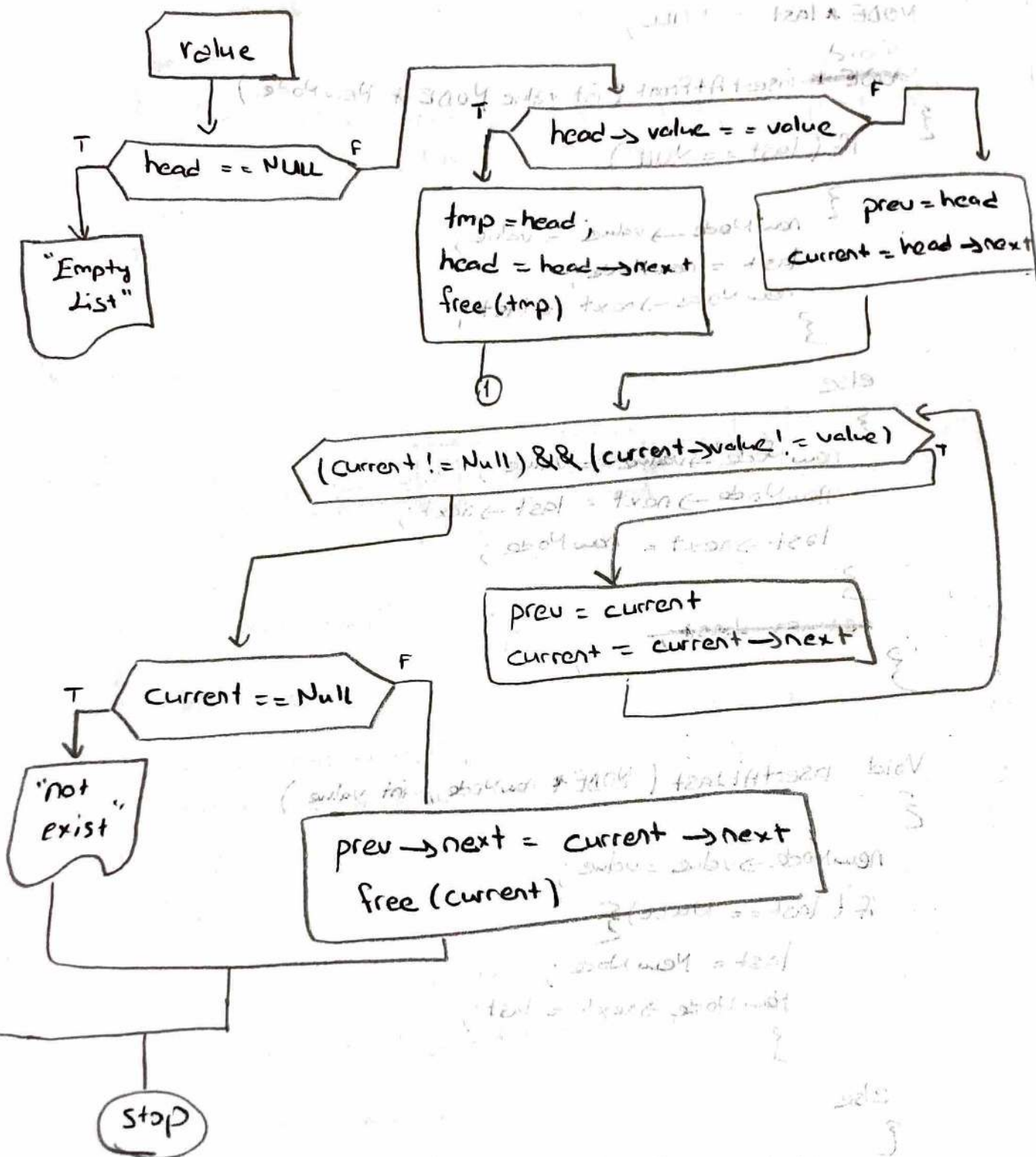
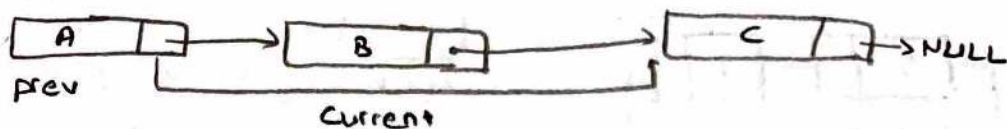
```

{
    NODE * ptr;
    ptr = head;
    while ( ptr != Null )
    {
        printf ("%d\n", ptr->value);
        ptr = ptr->next;
    }
}

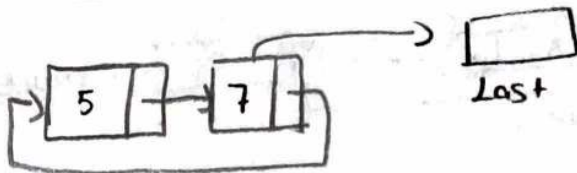
```


Delete in a Linked List

delete = 3



Circular Singly Linked List



NODE *last = NULL;

void ~~insertAtFront~~ insertAtFront (int value, NODE * NewNode)

```

{
    if (last == NULL)
    {
        newNode->value = value;
        last = newNode;
        newNode->next = last;
    }
  
```

else

```

{
    newNode->value = value;
    newNode->next = last->next;
    last->next = newNode;
}
return last;
}
  
```

Void insertAtLast (NODE * newNode, int value)

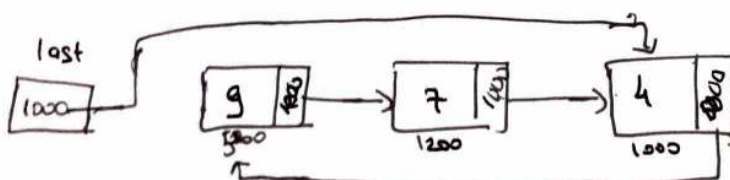
```

{
    newNode->value = value;
    if (last == NULL)
    {
        last = newNode;
        newNode->next = last;
    }
  
```

else

```

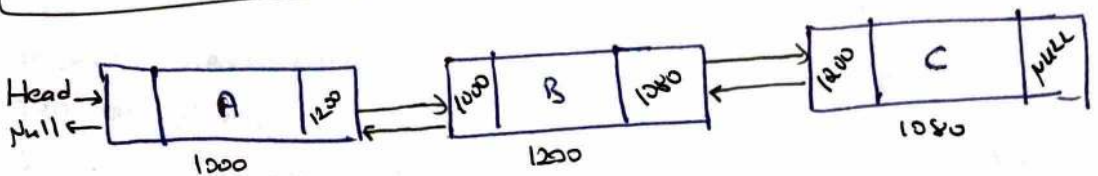
{
    newNode->next = last->next;
    last->next = newNode;
    last = newNode;
}
  
```



```
void PrintList()
```

```
{
    NODE *p;
    if (last == NULL)
        printf("List is empty");
    else
    {
        for (p = last->next; p != last->next; p = p->next)
            printf("%d\n", p->value);
    }
}
```

Double Linked List



```
typedef struct Node
```

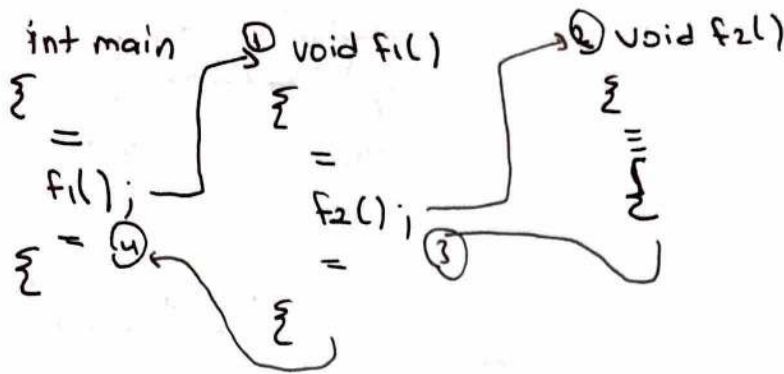
```
{
    int value;
    struct Node * next;
    struct Node * prev;
} NODE;
```

① Add a Node at the front

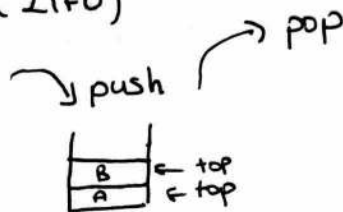
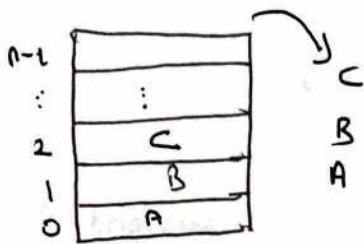
```
void insertAtFront (NODE * new Node, int value)
```

```
{
    new Node -> value = value;
    if (head == NULL)
    {
        head = new Node;
        head->prev = NULL;
    }
    else
    {
        head->prev = new Node;
        new Node->next = head;
        head = new Node;
    }
}
```


STACK (YİĞİN) → linear data structure



LAST IN FIRST OUT (LIFO)



push(A)
push(B)
x = pop() ⇒ x = B

#define MAXSIZE 100

typedef struct

{
int items[MAXSIZE];
int top;

} stack;

void initialize (STACK *s)

{
s->top = 0;

}

int empty (STACK *s)

{
if (s->top == 0)
return 1;

else
return 0;

}

int full (STACK *s)

{
if (s->top == MAXSIZE)
return 1;

else
return 0;

}

pop için
gerekli

push için
gerekli

①

int push (int x, STACK *s)

{
if (full(s))
return 0;

else

{
s->items[s->top] = x;
s->top++;
return 1;

}

int pop (STACK *s) int *x

{
if (empty(s))
return 0;

else

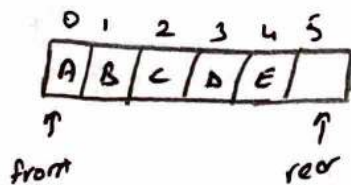
{
-- s->top;
*x = s->items[s->top];
return 1;

}

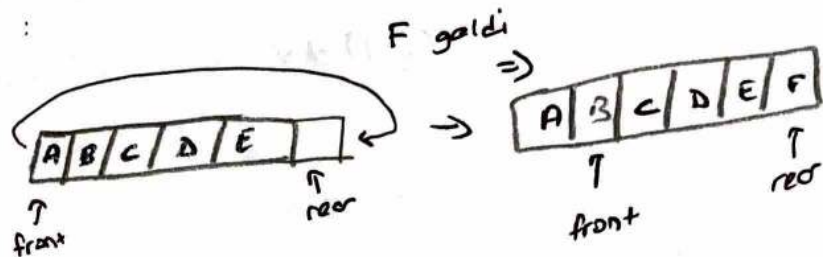
}

①

QUEUE : First In first Out (FIFO)



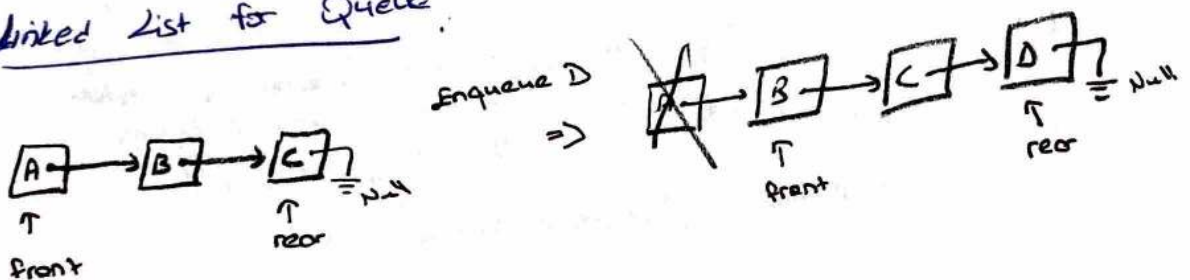
Circular Queue :



isFull, isEmpty ile kontrol et

- ① Enqueue : insert data
- ② Dequeue : remove data
- ③ Peek : get the element at the front

Linked List for Queue :



KOD

```
typedef struct QNode {
```

```
    int key;
```

```
    struct QNode * next;
```

```
} QNode;
```

```
typedef struct Queue {
```

```
    QNode * front, * rear;
```

```
} Queue;
```

Queue * createQueue ()

```
{  
    Queue * q = (Queue *) malloc (sizeof (Queue));  
    q → front = q → rear = Null;  
    return q;  
}
```

void enqueue (Queue * q, int key) {

```
    QNode * tmp = newNode (key);  
    if (isEmpty (q)) {  
        q → front = q → rear = tmp;  
    }
```

else {

```
    q → rear → next = tmp;  
    q → rear = tmp;  
}
```

}

~~void~~ dequeue (Queue * q, ~~int key~~) {
 QNode * tmp = q → front; int key;

if (isEmpty (q)) {

printf ("Queue is empty"); return -1;

}

```
else { key = q → front → key;  
    q → front = q → front → next;
```

```
    free (tmp);  
    return key;  
    } if (q → front == Null)  
        q → rear = Null;
```

}

QNode * newNode (int key) {

```
    QNode * tmp = (QNode *) malloc (sizeof (QNode));
```

```
    tmp → key = key;
```

```
    tmp → next = Null;
```

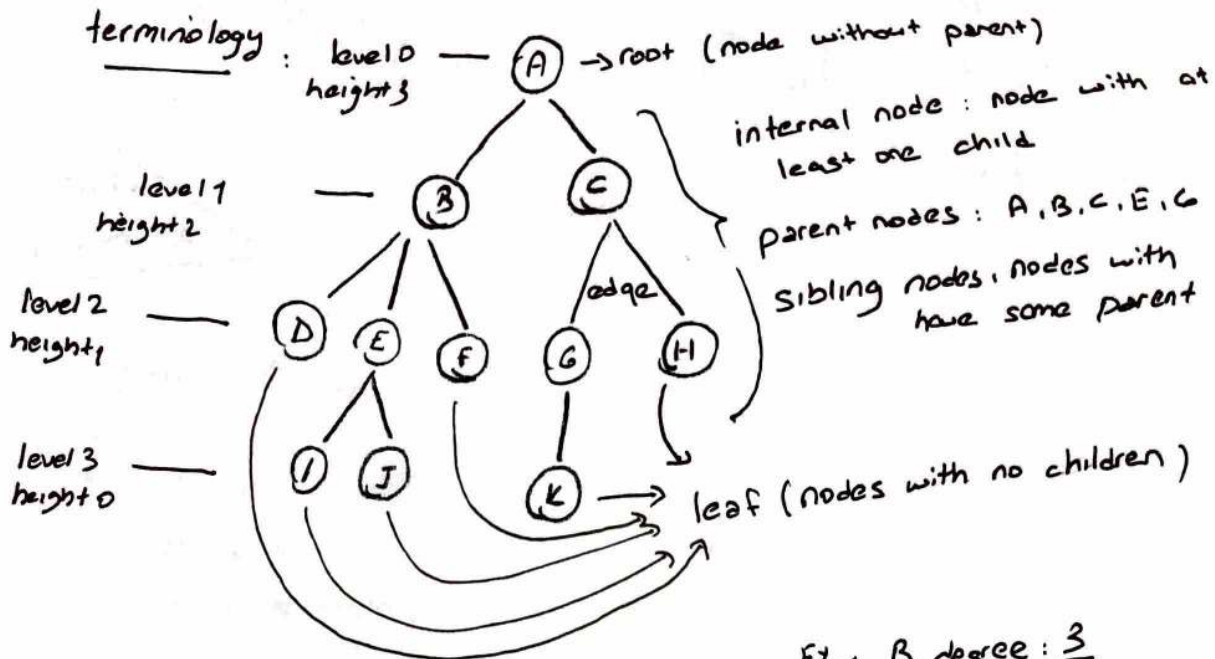
```
    return tmp;
```

}

TREE

→ non-linear data structure

terminology :

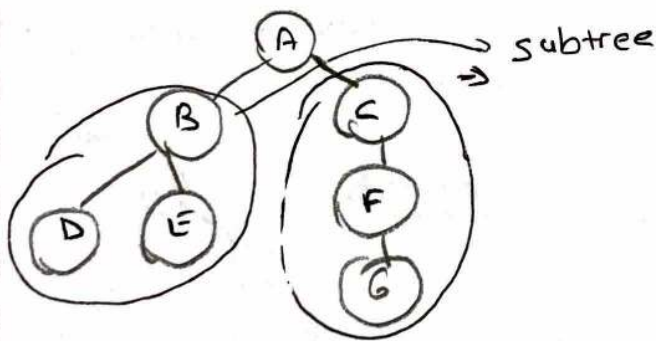


Degree : the total number of children $\text{Ex } B \text{ degree} : \underline{\underline{3}}$

Level : resül, (yaprığın kökten uzaklığı) (depth)

Height : node'un yapraklara uzaklığı

Path between A and E : A-B-E

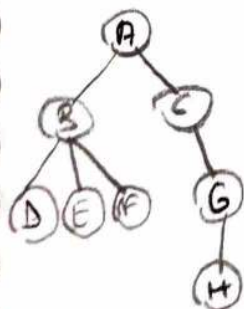


Tree Representation

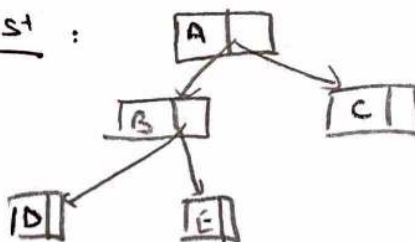
① Array ⇒

0	1	2			
A	B	C		D	E

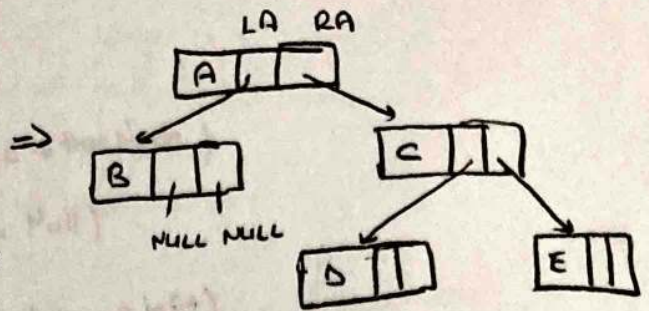
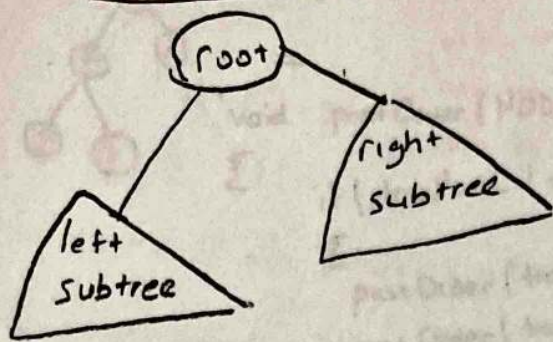
 = efektif değil



② Linked List :



Binary Tree :



struct BNode

```

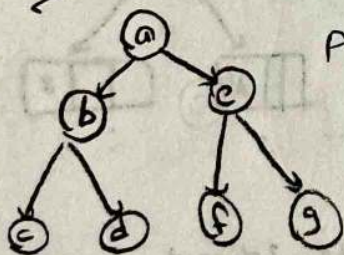
{
    int value;
    BNode * left;
    BNode * right;
}
  
```

⇒ Preorder traversal: Node → left → right + a b prefix

⇒ Inorder traversal: left → node → right a + b infix

⇒ Postorder traversal: left → right → node a b + postfix

Ques



preorder traversal = Node → left → right

=

typedef struct

```

{
    int value;
    struct Node * left, * right;
} node;
  
```

void printPreorder (Node * treeNode) {

if (treeNode != Null) {

printf ("Node\n", treeNode → value);

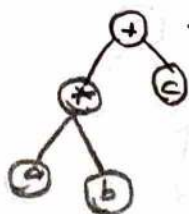
printPreorder (treeNode → left);

printPreorder (treeNode → right);

}

Postorder Traversal

Left - Right - Root



=
void postOrder (NODE *treeNode)

{ if (treeNode != Null)

{ postOrder (treeNode -> left);

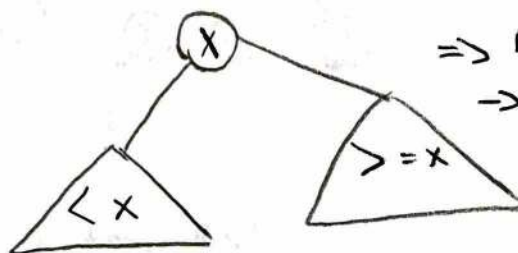
postOrder (treeNode -> right);

printf (" %d \n", treeNode -> value);

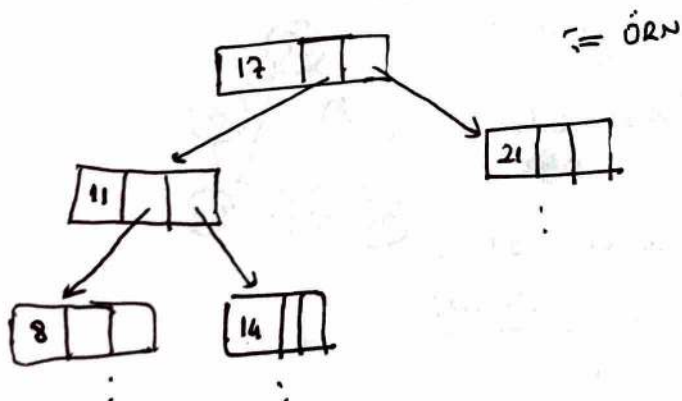
}

}

Binary Search Tree



=> BST morigi
-> divide and conquer



= ORN

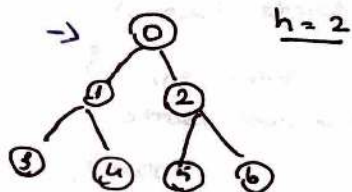
=
typedef struct node

{ int value;

struct node *left, *right;

} NODE;

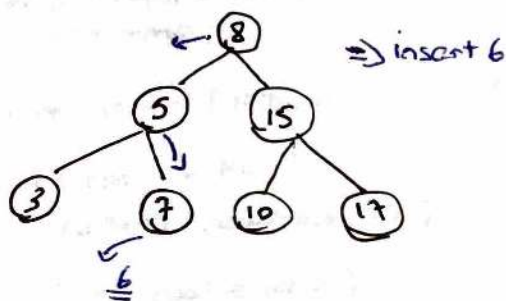
Binary tree of height (h)



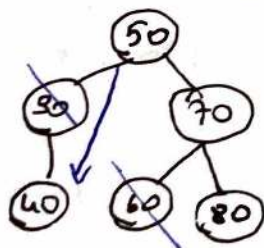
$\max \# \text{ of leaves} = 2^h \quad 4$
 $\max \# \text{ of nodes} = 2^{(h+1)} - 1 \quad 7$
 $\min \# \text{ of leaves} = 1$
 $\min \# \text{ of nodes} = h+1 \quad 3$

n node den bir ağacın yüksekliği $\Rightarrow \underline{O(\log_2 n)}$ (Ağacın dengeli ise)
 dengeli ağacın aranma zamanı $O(\log_2 n)$ karmaşıklıkta bulabiliriz.

Insertion of a key:



Deletion from BST



1. Node to be deleted is leaf
 $\times 60$

2. Node to be deleted has only one child:
 $\times 30$

3. Node to be deleted has two child

delete 50, sağ alt ağacın en küçük elemanı
 köklerde problem tekrar çözülür.
60 $\rightarrow$ yeni kök

=

```
typedef struct node
```

```
{  
    int value;  
    struct node *left, *right;
```

```
} NODE;
```

```
NODE * newNode (int x)
```

```
{  
    NODE * tmp;  
    tmp = (NODE *) malloc (sizeof (NODE));  
    tmp → value = x;  
    tmp → left = null;  
    tmp → right = null;  
    return tmp;
```

```
NODE * insert (NODE * root, int x)
```

```
{  
    if (root == Null)  
        return newNode (x);  
  
    if (x < root → value)  
        root → left = insert (root → left, x);  
  
    else  
        root → right = insert (root → right, x)
```

```
    return root;
```

```
}
```

```
int main ()
```

```
{  
    NODE * root = Null;  
    root = insert (root, 15);  
    insert (root, 17);  
}
```

```
NODE * search (NODE * root, int x)
```

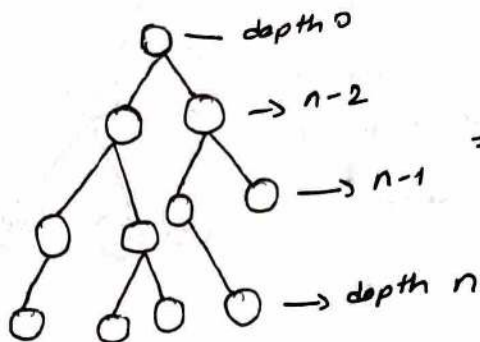
```
{  
    if ((root == Null) || (root → value == x))  
        return root;  
  
    else if (x < root → value)  
        return search (root → left, x);  
  
    else  
        return search (root → right, x);
```

```
}
```

Heap Tree

→ balanced, left justified binary tree, no node has a value greater than the value of its parents.

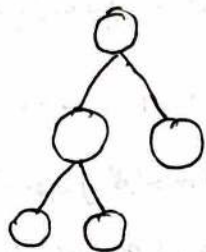
① Balanced:



→ from depth 0 to depth $n-2$, all nodes have two children

⇒ BALANCED

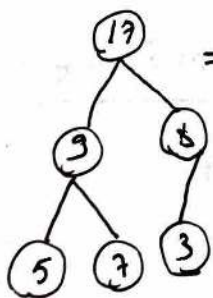
② Left Justified Binary Tree



→ yaprakların sola dayalı olduğu binary tree

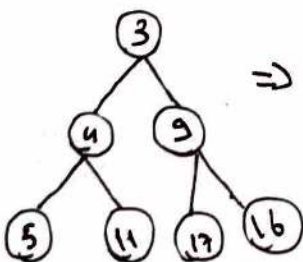
= Left Justified

③ Heap Property



⇒ Max Heap

: her zaman parentların çocuklarından büyük olduğu tree.

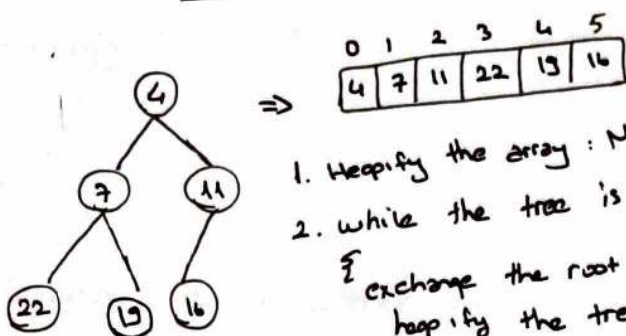


⇒ Min heap: her zaman çocukların parentlardan büyük olduğu tree.

→ Heap Treelerde diziyi saklamak yapılabilir.

HeapSort Algorithm

Complexity $\rightarrow \underline{n \log N}$



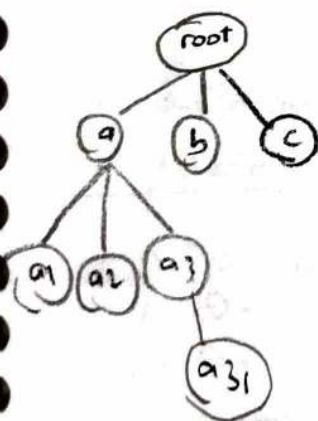
1. Heapify the array : N
2. while the tree is not empty
 - { exchange the root with last node
 - heapify the tree $\rightarrow \log N$

$N \cdot \log N$

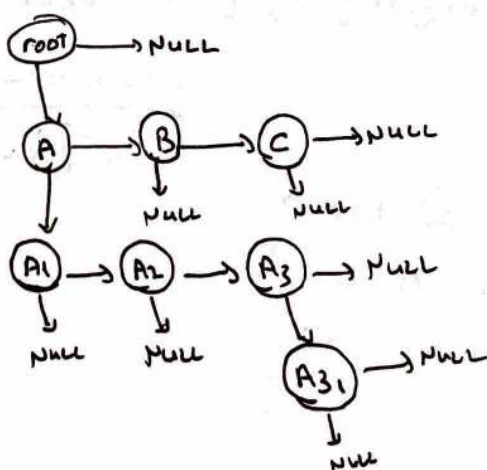
```

void heapSort(int Heap[], int n)
{
    for (i = n-1; i > 0; i--)
    {
        swap(&Heap[0], &Heap[i]);
        siftDown(Heap, i); // heapify
    }
}
  
```

Binary olmajor Tree'ler



First child / next sibling kullonlu

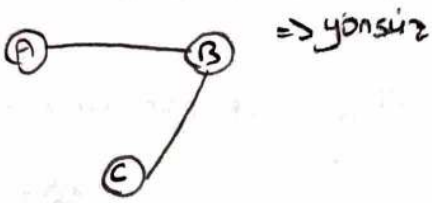


```

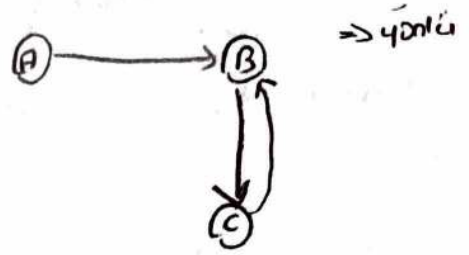
=
struct Node
{
    char name[40];
    struct NODE * firstChild;
    struct NODE * nextSibling;
} TREE;
  
```

Graphs

Undirected Graphs



Directed Graphs

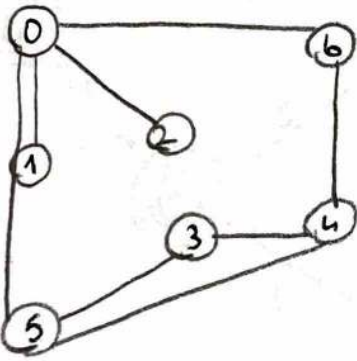


$\Rightarrow$ Bilgisayar ortamına dökmek için Symbol Table oluşturuldu.

Undirected Graph için,

- $\Rightarrow$ komşuluk matrisi kullanılır.

$$adj[v][w] = adj[w][v] = true$$



	0	1	2	3	4	5	6
0	0	1	1	0	0	1	1
1		0	0	0	0	0	0
2		1	0	0	0	1	0
3		0	0	0			
4				1		1	1
5		1		1	1		
6		1			1		

1. gösterim

$\Rightarrow$ Bağlantının ağırlığı varsa $\rightarrow$ ağırlığı yazıyoruz.

2. gösterim : Linkli Liste yapısı

- 0: 5 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 6 .
1: 0 .
2: 0 .
3: 4 $\rightarrow$ 5 .
4: 6 $\rightarrow$ 5 $\rightarrow$ 3 .
5: 0 $\rightarrow$ 3 $\rightarrow$ 4 .
6: 4 $\rightarrow$ 0 .

```
struct vertex  
{  
    int number;  
    struct vertex *next;  
};
```

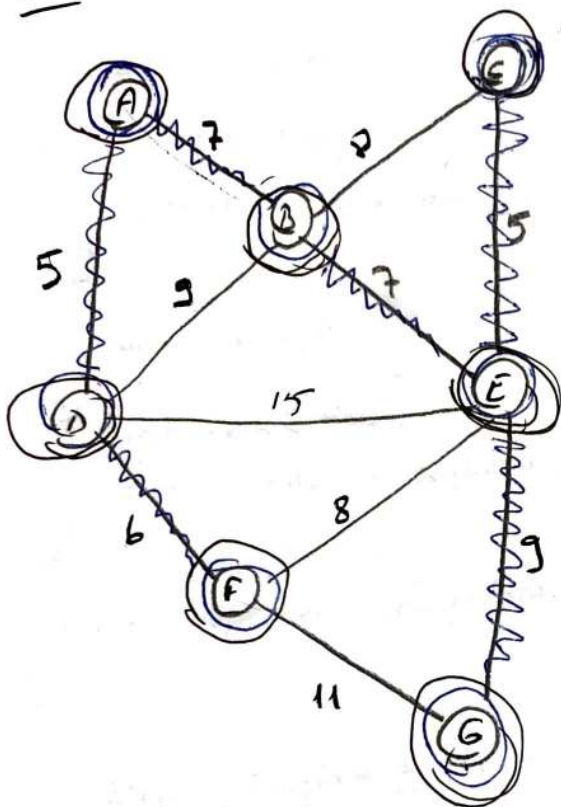
```
struct graph  
{  
    int nvertices;  
    struct vertex **adjList;  
};
```

Minimum Spanning Tree

→ undirected, acyclic graph / contains all the vertices in graph

→ spanning tree with the minimum sum of weights

Ques



Prim's Algorithm

Greedy Algorithm

→ a locally optimal choice is globally optimal

$$AD + DF + AB + BE + EC + EG = 39$$

Kruskal Algorithm

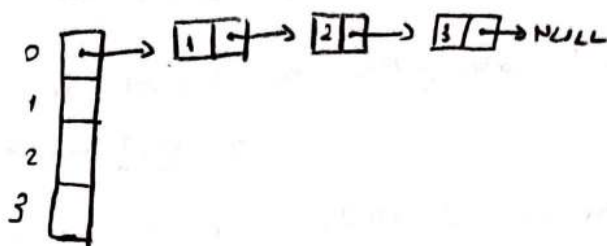
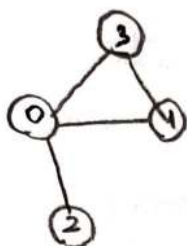
- AD → 5 ✓
- CE → 5 ✓
- DF → 6 ✓
- AB → 7 ✓
- BE → 7 ✓
- BC → 8 ✗
- EF → 8 ✗
- BD → 9 ✗
- EG → 9 ✓
- FG → 11
- DE → 15

$n-1$ edges →

$$= 39$$

A	B	C	D	E	F	G
0	1	2	3	4	5	6
1	1	2	1	2	1	1
		1		1		

Konsultasi Listasi in C



```
=
struct node
{
    int vertex;
    struct node * next;
};

struct Graph
{
    int numVertices;
    struct node ** adjLists;
};
```

```
struct Graph * createGraph(int vertices)
{
    int i;
    struct Graph * graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct node *));
    for (i = 0; i < vertices; i++)
        graph->adjLists[i] = NULL;
    return graph;
};
```

```
void addEdge(struct Graph * graph, int src, int dest)
{
    struct node * newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
};
```

```
struct node * createNode(int v)
{
    struct node * newNode = malloc(sizeof(struct node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
};
```

Kruskal Algorithm implementasyon

case 1 ikisi de herhangi bir alt grapha bağlı değil.

$$\text{label}[u] = \text{label}[v] = 0$$

labelNo arttırılıp u ve v labelNo ile gösterilen yeni bir altgrapha eklemeli.

case 2

$$\text{label}[u] \text{ veya } \text{label}[v] = 0 \text{ ise ;}$$

hangisi 0 ise diğerinin etiketi ile etiketlenmeli

case 3

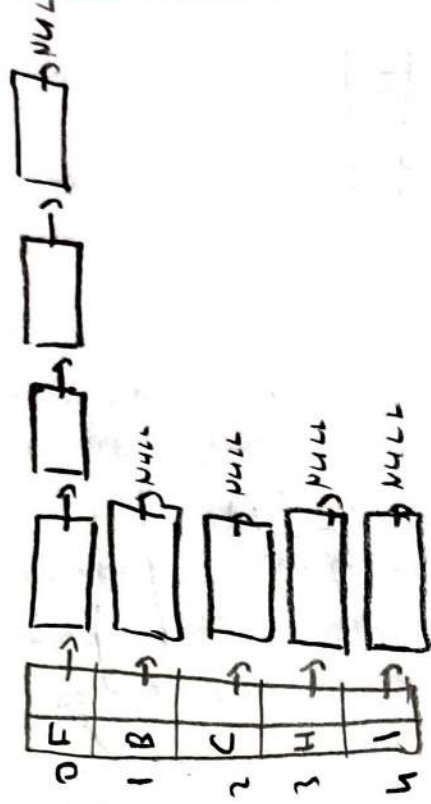
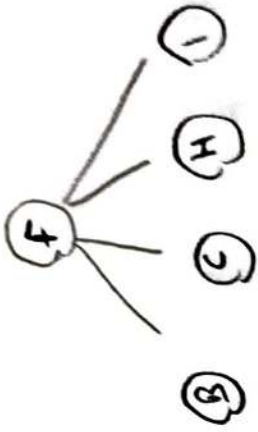
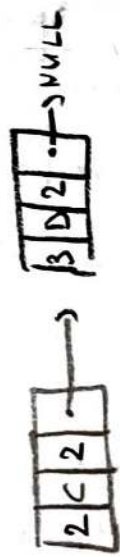
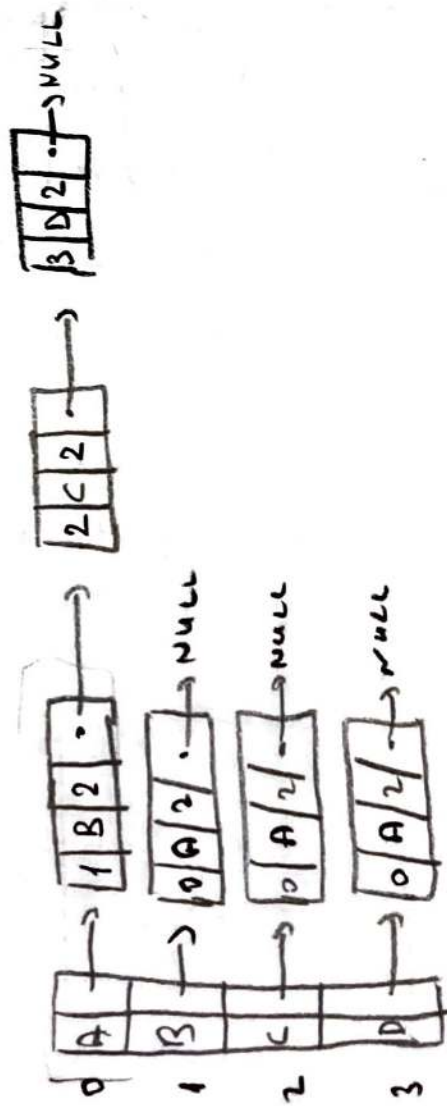
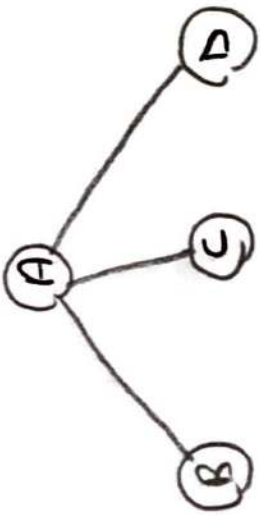
$$\text{label}[u] \neq \text{label}[v]$$

birisi ve aynı etiketli tüm düğümler diğerinin etiketi ile etiketlensin

case 4

$$\text{label}[u] = \text{label}[v]$$

işlem yapma, cycle oluşur.



11/5 → null